Project Proposal

VLSI Lab: CMOS Implementation of a 4-bit Datapath | Track A: 4-bit ALU

1. Project Title

Design, CMOS Schematic Implementation, Symbol Creation and Testbench Verification of a 4-Bit ALU (Addition and Subtraction) in Cadence Virtuoso

Course / Section	Track	Operations (this phase)	Submission
CSE-450, Section E1	A: 4-bit ALU (datapath component)	ADD, SUBTRACT (1-bit opcode)	12 October 2026

2. Introduction

A datapath is the part of a digital processor that holds, moves and transforms data. It is built from storage (registers or memory), selection logic (multiplexers) and an arithmetic logic unit (ALU), and it is steered by control signals generated by a control unit. Datapath design is central to digital and VLSI systems because the datapath sets most of a processor's speed, area and power; in particular, the carry path through the ALU is usually the longest combinational path and therefore limits the maximum clock frequency.

Of the four datapath components offered in this lab (ALU, register set, 16×4 memory multiplexer and control unit), this project implements **Track A, the 4-bit ALU**. As directed by the instructor, this phase implements exactly **two opcode-selected operations: addition and subtraction**. The ALU takes two 4-bit operands A and B and a 1-bit opcode OP, and produces a 4-bit result R and one overflow/error bit OVF. Subtraction reuses the adder through two's complement (A − B = A + B' + 1), so both operations share a single ripple-carry datapath built from four identical 1-bit slices.

The purpose of the project is to take this ALU through the complete Cadence workflow required by the lab: derive the output equations, decompose them into static CMOS gates with stated transistor sizes, draw the transistor-level schematic in Virtuoso, create a symbol for each level of the hierarchy, and verify the design with a separate testbench of at least ten documented test cases using transient simulation in ADE.

3. Problem Statement

A processor datapath needs a small combinational unit that can add or subtract two 4-bit words under the control of a select signal and report when the 4-bit result is not valid. The problem addressed here is to design such a unit at CMOS transistor level in Cadence Virtuoso: a 4-bit ALU whose 1-bit opcode selects A + B or A − B, whose 4-bit result is correct modulo 16, and whose overflow/error bit flags an unsigned carry-out during addition or a borrow (A < B) during subtraction. The design must use static CMOS cells powered from VDD and GND, expose clearly named pins on a reusable symbol, and be verified by simulation against expected outputs calculated before simulation, including boundary cases such as all-ones operands, zero results and results that wrap around.

4. Objectives

- Derive the Boolean equations of the result bits R3–R0, the internal carries C1–C4 and the overflow/error bit OVF for both operations (ADD and SUB).
- Decompose the equations into static CMOS cells (inverter, NAND2, XOR2) and document transistor sizes, supply voltage, load capacitance and input transition assumptions.
- Design and verify a transistor-level full adder and a 1-bit add/subtract slice (XOR-controlled B inversion followed by a full adder).
- Chain four slices into a 4-bit ripple-carry adder/subtractor and add the OVF logic.

- Create a symbol for every hierarchy level and a separate testbench cell for the full adder, the slice and the 4-bit ALU.
- Apply at least 10 test cases (13 are planned) covering both opcodes and the boundary conditions in ADE transient analysis, and compare actual waveforms with expected outputs.
- Measure the worst-case propagation delay of the carry-ripple path and the average power drawn from VDD, and report the transistor count as an area estimate.

5. Proposed System

5.1 Interface specification

The pin assignment follows the Track A signal table of the lab guideline (9 input bits, 5 output bits). Each guideline field is mapped to a named pin on the Cadence symbol.

Guideline field	Symbol pin	Width	Dir.	Description
input[0]	OP	1	In	Operation select: 0 = ADD, 1 = SUB
input[1:4]	A<3:0>	4	In	First operand (A<3> is the MSB)
input[5:8]	B<3:0>	4	In	Second operand (B<3> is the MSB)
output[0:3]	R<3:0>	4	Out	ALU result (R<3> is the MSB)
output[4]	OVF	1	Out	Overflow/error bit: carry-out for ADD, borrow for SUB
–	VDD, GND	1 each	Power	Supply rails (1.8 V and 0 V)

5.2 Operation table

OP	Operation	Equation	C0	Adder's B input	OVF = 1 means
0	Addition	R = A + B	0	B (unchanged)	Carry-out: A + B > 15
1	Subtraction	R = A − B = A + B' + 1	1	B' (inverted)	Borrow: A < B

5.3 Output equations

For each bit i = 0 to 3 (one slice per bit):

$$B'_i = B_i \oplus OP \qquad C_0 = OP$$
$$P_i = A_i \oplus B'_i \quad \text{(propagate)}$$
$$R_i = S_i = A_i \oplus B'_i \oplus C_i = P_i \oplus C_i$$
$$C_{i+1} = A_i \cdot B'_i + C_i \cdot P_i = [\,(A_i \cdot B'_i)' \cdot (C_i \cdot P_i)'\,]'$$
$$OVF = C_4 \oplus OP$$

Written out bit by bit (as in the guideline's ripple-carry example, but with B' and C0 = OP so that the same equations cover subtraction):

Bit	Result bit	Carry out of the bit
0	R0 = A0 ⊕ B'0 ⊕ OP	C1 = A0·B'0 + OP·(A0 ⊕ B'0)
1	R1 = A1 ⊕ B'1 ⊕ C1	C2 = A1·B'1 + C1·(A1 ⊕ B'1)
2	R2 = A2 ⊕ B'2 ⊕ C2	C3 = A2·B'2 + C2·(A2 ⊕ B'2)
3	R3 = A3 ⊕ B'3 ⊕ C3	C4 = A3·B'3 + C3·(A3 ⊕ B'3)
Flag	OVF = C4 ⊕ OP	C4 is the carry out of slice 3

Why OVF = C4 ⊕ OP. In ADD mode OVF equals the carry-out C4, which is the guideline's “carry from addition”. In SUB mode C4 = 1 means no borrow occurred (A ≥ B), so XOR-ing with OP inverts it and OVF = 1 flags a borrow. In both modes OVF = 1 therefore means the 4-bit unsigned result is not the true answer. A single XOR2 implements it. The carry form $A_iB'_i + C_iP_i$ is equivalent to the majority form $A_iB'_i + A_iC_i + B'_iC_i$ used in the guideline, but it reuses P_i from the sum path and maps directly onto three NAND2 gates.

5.4 Architecture

Figure 1 shows the top level. OP drives the B-inversion XOR in every slice and is also the carry-in C0 of slice 0, which supplies the “+1” of two's complement during subtraction. Four identical slices are connected by a ripple-carry chain C1–C4, and one XOR2 forms OVF from C4 and OP. Because only two operations are implemented, the full-adder sum is the result bit directly and no output multiplexer is needed.

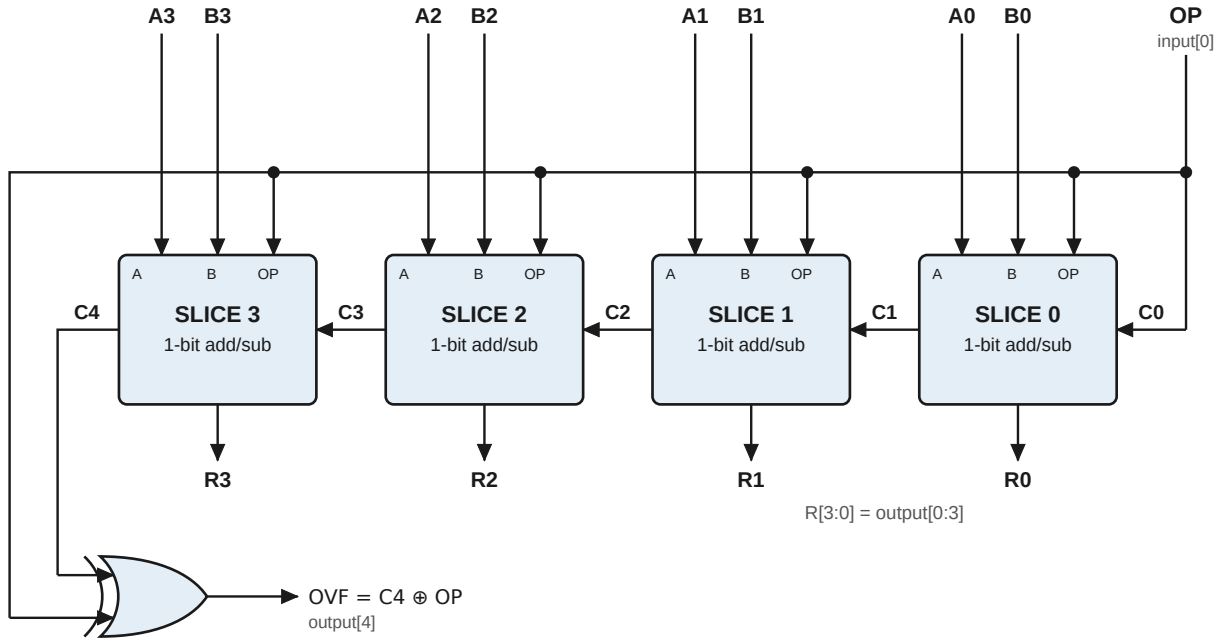


Figure 1. Top-level block diagram of the 4-bit ALU (addition/subtraction)

Figure 2 shows the gate-level structure of one slice, which is the logic decomposition that will be drawn at transistor level. The first XOR2 conditionally inverts B_i; the full adder is made of two XOR2 gates on the sum path and three NAND2 gates (NAND-NAND form) on the carry path.

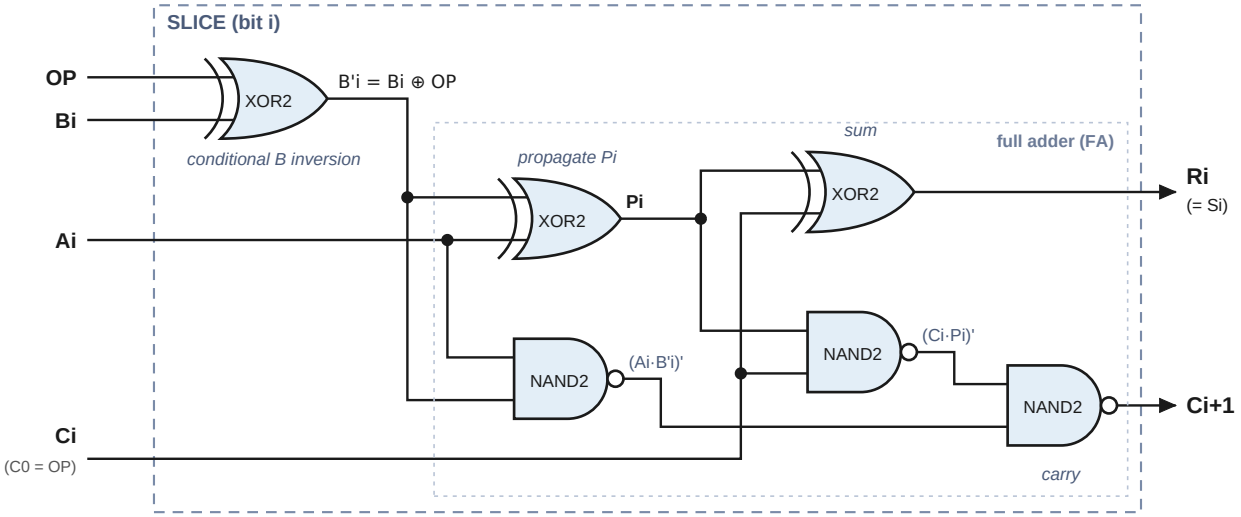


Figure 2. Gate-level structure of one 1-bit add/subtract slice (for slice 0, Ci is tied to OP)

5.5 Logic decomposition and transistor sizing assumptions

All cells are static complementary CMOS. Every transistor uses the minimum channel length of the technology. NMOS widths are doubled for two-transistor series stacks, and PMOS widths are about twice the NMOS width to compensate for lower hole mobility, so each cell has roughly the drive strength of the reference inverter.

Cell (hierarchy level)	Function	CMOS implementation	Count	W/L (L = 180 nm)
INV	Y = A'	1 PMOS + 1 NMOS	2	Wp = 2 µm, Wn = 1 µm
NAND2	Y = (A·B)'	2 parallel PMOS, 2 series NMOS	4	Wp = 2 µm, Wn = 2 µm
XOR2	Y = A ⊕ B	Complementary network (AB + A'B')' with 2 INV for A', B'	12	Network: Wp = 4 µm, Wn = 2 µm; INV as above
FA (full adder)	Si, Ci+1	2 XOR2 + 3 NAND2	36	From sub-cells
SLICE	1-bit add/sub	1 XOR2 (B inversion) + FA	48	From sub-cells
ALU4 (top)	4-bit ADD/SUB	4 SLICE + 1 XOR2 (OVF)	204	From sub-cells

Parameter	Assumed value
Technology	gpdk180 (Cadence 180 nm generic PDK), NMOS/PMOS models from the kit
Supply	VDD = 1.8 V, GND = 0 V; NMOS bulk to GND, PMOS bulk to VDD
Load capacitance	10 fF on each output pin R<3:0> and OVF in the testbench
Input transitions	Rise/fall time 100 ps on all input sources
Temperature / corner	27 °C, typical-typical (TT) models

OP has a fan-out of six (four B-inversion XORs, C0 and the OVF XOR). If simulation shows a slow OP edge, a two-inverter buffer will be added on OP.

5.6 Implementation flow in Cadence

The design follows the lab's mandatory Cadence workflow, bottom-up for each cell (INV, NAND2, XOR2, FA, SLICE, ALU4): functional specification (5.1–5.2) → equations (5.3) → logic decomposition and sizing (5.5) → transistor-level schematic in Virtuoso → **Check and Save** with zero errors and warnings (no floating inputs or unconnected nets, every NMOS bulk tied to GND and every PMOS bulk tied to VDD) → symbol created from the schematic with the same pin names → separate testbench cell instantiating the symbol → at least 10 test cases → transient simulation in ADE → waveforms compared with the expected table → report and viva. No transistor is drawn before its equation is documented.

6. Tools and Technologies

Tool / Technology	Purpose
Cadence Virtuoso Schematic Editor	Transistor-level and hierarchical schematic entry for INV, NAND2, XOR2, FA, SLICE and ALU4
Virtuoso Symbol Editor	Symbol creation for each cell with pins OP, A<3:0>, B<3:0>, R<3:0>, OVF, VDD, GND
Cadence ADE (Spectre simulator)	Transient analysis of the testbench cells using vdc, vpulse and vpwl sources
Virtuoso Visualization & Analysis (calculator)	Waveform viewing, 50%-to-50% delay measurement and average-power calculation
gpdk180 process design kit	NMOS/PMOS device models and design rules used by the schematics
Python script / Verilog model (optional)	Behavioural reference to generate the expected-output table for comparison
Word / LaTeX, screenshots	Proposal, final report with schematics, symbols and waveforms

7. Expected Outcome

- Complete transistor-level schematic of the 4-bit ADD/SUB ALU with INV, NAND2, XOR2, full-adder and slice sub-cells.

- A symbol for each hierarchy level, with the top-level symbol pins matching the Track A interface in Section 5.1.
- Separate testbench cells for the full adder, the 1-bit slice and the 4-bit ALU.
- Transient simulation waveforms showing OP, A<3:0>, B<3:0>, all four result bits and OVF for every test case.
- Verified outputs: a completed expected-vs-actual table with pass/fail for every case.
- Block diagrams, gate-level diagrams and output equations (Section 5).
- Performance figures: worst-case carry-path delay, average power from VDD and transistor count.
- Synthesis results from the optional Verilog model, if required by the instructor.
- Final project report and viva demonstration.

8. Testing and Verification Plan

Module-level testing. Each cell is verified in its own testbench before it is used in the next level. INV, NAND2 and XOR2 are tested with all input combinations. The full adder is tested with all 8 combinations of A_i, B'_i, C_i. The slice is tested with all 16 combinations of A_i, B_i, C_i and OP. The 4-bit ALU is then tested with the 13 directed cases below; if time permits, all 512 combinations (256 operand pairs for each opcode) are swept with binary-weighted vpulse sources and compared with the Python reference. Because the ALU4 input space (512 combinations) is large, the directed cases are backed by the exhaustive slice test, which covers every input pattern one bit position can see.

Stimulus and timing. In the ALU4 testbench each case is applied with vpwl sources and held for 10 ns, so 13 cases occupy 0–130 ns of a 140 ns transient run. Expected values are calculated before simulation (table below), and each 10 ns window is marked on the waveform screenshots so the actual output can be read at the end of the window and entered in the table.

#	Test case	OP	A	B	Expected R	Expected OVF	Actual R / OVF	Pass/ Fail
1	Normal add (5 + 3)	0	0101	0011	1000	0		
2	Zero + zero	0	0000	0000	0000	0		
3	Carry ripples bits 0–2 (7 + 1)	0	0111	0001	1000	0		
4	No carry, all propagate (10 + 5)	0	1010	0101	1111	0		
5	Overflow discarded (15 + 1)	0	1111	0001	0000	1		
6	Boundary: max + max (15 + 15)	0	1111	1111	1110	1		
7	Normal subtract (7 − 2)	1	0111	0010	0101	0		
8	Same operands as case 1 (5 − 3)	1	0101	0011	0010	0		
9	Equal operands (7 − 7)	1	0111	0111	0000	0		
10	Subtract zero (15 − 0)	1	1111	0000	1111	0		
11	Borrow ripples bit 0 → bit 3 (8 − 1)	1	1000	0001	0111	0		
12	Negative result (3 − 5)	1	0011	0101	1110	1		
13	Boundary: 0 − 1 wraps to 1111	1	0000	0001	1111	1		

Notes. Cases 1, 7 and 5 are the guideline's short test cases (5 + 3, 7 − 2, 15 + 1). Cases 1 and 8 apply the same operands under both opcodes to confirm that OP alone changes the operation. Boundary cases cover zero, all-ones, equal operands and wrap-around in both directions. The OVF column is checked for both meanings: carry-out in ADD (cases 5 and 6) and borrow in SUB (cases 12 and 13). As a quick sanity check, R0 equals A0 ⊕ B0 in every case, for both operations.

Clock and reset. The ALU is purely combinational, so it has no clock or reset input; correct operation is judged by the settled output in each 10 ns window. Clock and reset behaviour will only apply if the ALU is later registered inside a complete datapath.

Delay and power. The worst-case delay is measured on the carry-ripple path: with A = 1111, B is switched from 0000 to 0001 in ADD mode so the carry travels from slice 0 to C4, and the 50%-to-50% delay from B0 to R3 and to OVF is recorded. The same is repeated by switching OP from 0 to 1 with A = B = 0000. Average power is computed as VDD × the average supply current over the full 13-case run.

9. Team Responsibility

Member (Name, ID)	Responsibility
Md Abdullah Al Jobayer (22201245)	Transistor-level INV, NAND2 and XOR2 cells with sizing; full-adder schematic, symbol and testbench (8 cases)
H. M. Tahsin Sheikh (22201243)	1-bit add/subtract slice (B-inversion XOR + full adder), slice symbol and testbench (16 cases)
Shahrier Shimul (22201227)	4-bit integration, OVF logic, top-level symbol, ALU4 testbench with the 13 test cases, delay and power measurement
All members	Equation derivation, expected-output table, waveform verification, final report and viva demonstration